

CS 61B — Data Structures

Exam-Level Study Guide

Course: Computer Science 61B — Data Structures

Language: Java

Course site: <https://sp26.datastructur.es/>

Final exam: Tue May 12, 2026 — exact time/location: verify at <https://sp26.datastructur.es/>

Format: In-person, written. Typically 180 min. Multiple handwritten cheat sheets allowed (printed-from-tablet are confiscated).

How to use this guide. Each topic block follows the same shape:

- **Definition** — formal one-line statement.
- **Plain English** — what it actually means.
- **Invariant / API / pseudocode** — the canonical reference (cheat-sheet candidates).
- **Runtime** — Big- Θ for each operation. Memorize these tables cold.
- **Trace example or diagram** — concrete worked example.
- **Exam patterns / pitfalls** — what 61B finals do with this concept.

Calibration note. You already have `MT2_study_guide.md` and `MT2_cheatsheet.md` — those cover roughly half of this guide (asymptotics through heaps). This guide is the structured catalog over the entire course; for MT2-territory topics you should rely on your existing sheets first and use this only for cross-referencing.

Table of Contents

1. Java Fundamentals — References, Primitives, Static vs Instance
 2. Testing
 3. Lists — IntList → SLList → DLList → AList → ArrayList
 4. Inheritance — extends, super, equals, hashCode, iterators
 5. Asymptotics — Big-O / Ω / Θ
 6. Recurrences and Master Theorem
 7. Amortized Analysis
 8. Disjoint Sets (Union-Find)
 9. Binary Search Trees (BSTs)
 10. B-Trees (2-3-4 Trees)
 11. Tree Rotations and Red-Black Trees
 12. Heaps and Priority Queues
 13. Hashing
 14. Tries
 15. Graphs — Representations and Traversals (DFS, BFS)
 16. Shortest Paths — Dijkstra, A*
 17. Minimum Spanning Trees — Prim, Kruskal
 18. Directed Acyclic Graphs and Topological Sort
 19. Sorting — Basic Sorts, Quicksort, Mergesort
 20. Sorting — Theoretical Bounds, Radix Sort
 21. Compression (Huffman) — light coverage
-

1. Java Fundamentals — References, Primitives, Static vs Instance

[Lec 1–3]

Primitive vs Reference Types

Definition. Primitives (`int`, `double`, `boolean`, `char`, `long`, `byte`, `short`, `float`) hold values directly. References hold a pointer to an object on the heap.

Plain English. `int x = 5` puts the bits 5 in `x`. `int[] a = new int[3]` puts a *pointer* to a heap-allocated array in `a`. Assigning a reference does NOT copy the object.

The Golden Rule of Equals (Java).

- Primitives: `==` compares values.
- References: `==` compares pointers (same object?). Use `.equals()` for value equality.

```
int x = 5, y = 5;
x == y           // true (value comparison)

int[] a = {1, 2}, b = {1, 2};
a == b           // false (different objects)
java.util.Arrays.equals(a, b) // true

String s1 = "hi", s2 = "hi";
s1 == s2         // typically true due to string interning — DON'T RELY ON THIS
s1.equals(s2)    // true (always)
```

Static vs Instance

- **Instance variable / method.** Belongs to a specific object. Accessed via `obj.field`.
- **Static variable / method.** Belongs to the class itself. Accessed via `ClassName.field`. There is exactly ONE copy across all instances.
- **Inside a static method**, you cannot reference `this` (no instance exists).

```
public class Dog {
    public static int numDogs = 0;    // shared across all Dogs
    public String name;               // each Dog has its own

    public Dog(String n) {
        name = n;
        numDogs += 1;
    }
}
```

```
public static int getCount() {      // static method
    return numDogs;
}
}
```

null and NullPointerException

Definition. `null` is a special reference value meaning "points to nothing."

Plain English. Calling `.method()` on `null` throws `NullPointerException` (NPE) at runtime. The #1 bug source in Java.

Exam patterns / pitfalls

- "Will this code compile? If so, what does it print?" — common trace question; watch for `static` vs instance access errors.
 - `String s = null; s.length();` — NPE.
 - Arrays in Java are objects; `arr.length` is a field (no parens), `str.length()` is a method (parens).
-

2. Testing

[Lec 4]

Unit Tests with JUnit

Plain English. A test is a method that exercises some behavior and asserts the expected outcome. JUnit runs all `@Test`-annotated methods and reports pass/fail.

```
import static org.junit.Assert.*;
import org.junit.Test;

public class ListTest {
    @Test
    public void testAddOne() {
        AList<Integer> a = new AList<>();
        a.addLast(5);
        assertEquals(5, (int) a.get(0));
    }
}
```

Common assertions

- `assertEquals(expected, actual)`
- `assertTrue(condition)` / `assertFalse(condition)`
- `assertArrayEquals(expected, actual)` (for arrays)
- `assertNull(obj)` / `assertNotNull(obj)`

Exam patterns / pitfalls

- Rare on the final — usually conceptual ("why test before implementing?") at most.
-

3. Lists — IntList → SLList → DLList → AList → ArrayList

[Lec 5–8]

IntList (the "naked" recursive list)

Invariant. A list is either `null` (empty) or a node with a `first` `int` and a `rest` `IntList`.

```
public class IntList {
    public int first;
    public IntList rest;      // null if last node

    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }
}
```

Recursive operation example:

```
public static int size(IntList L) {
    if (L == null) return 0;
    return 1 + size(L.rest);
}
```

SLList (Singly-Linked List with sentinel)

Invariant. `SLList` wraps an `IntList`. Has a `sentinel` node always present (never `null`), making the empty case look like the general case.

```
public class SLList {
    private IntNode sentinel;      // always non-null
    private int size;

    private static class IntNode {
        int item;
        IntNode next;
        IntNode(int i, IntNode n) { item = i; next = n; }
    }

    public SLList() {
        sentinel = new IntNode(0, null);    // garbage value in sentinel
    }
}
```

```

        size = 0;
    }

    public void addFirst(int x) {
        sentinel.next = new IntNode(x, sentinel.next);
        size += 1;
    }

    public int size() { return size; }
}

```

Why sentinel? Eliminates the special "is the list empty?" branch in every operation. Cleaner code, fewer bugs.

DLList (Doubly-Linked List, circular sentinel)

Invariant. Each node has both `prev` and `next`. The sentinel's `next` is the first real node; sentinel's `prev` is the last real node. Circular structure makes `addLast` $O(1)$.

```

private class IntNode {
    IntNode prev;
    int item;
    IntNode next;
}

```

- `addLast(x)`: create node, splice between `sentinel.prev` and `sentinel`. $O(1)$.
- `removeLast()`: unlink `sentinel.prev`. $O(1)$.
- `get(i)`: walk i steps from `sentinel`. $O(n)$.

AList (Array-backed list with naive resize)

Invariant. Internally stores items in `int[] items`. Field `size` tracks how many slots are used.

```

public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100];
        size = 0;
    }

    public void addLast(int x) {
        if (size == items.length) resize(size * 2); // GEOMETRIC resize
    }
}

```

```

        items[size] = x;
        size += 1;
    }

    private void resize(int capacity) {
        int[] a = new int[capacity];
        System.arraycopy(items, 0, a, 0, size);
        items = a;
    }
}

```

Geometric resize is critical. Doubling the array on full-up gives **amortized $O(1)$** addLast. Adding one slot at a time would give $O(n)$ per addLast.

Operation runtime table (memorize)

Operation	SList	DList	AList
addFirst	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
addLast	$\Theta(n)$ (or $\Theta(1)$ with last pointer)	$\Theta(1)$	$\Theta(1)$ amortized, $\Theta(n)$ worst
removeFirst	$\Theta(1)$	$\Theta(1)$	$\Theta(n)$
removeLast	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
get(i)	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$
size()	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

Exam patterns / pitfalls

- "Trace SList after these calls" — draw nodes, sentinel, pointers. Sentinel always present.
- "Why does AList double instead of grow by 100?" — to achieve amortized $O(1)$.
- Forgetting to update `size` on add/remove operations.

4. Inheritance — extends, super, equals, hashCode, iterators

[Lec 9–11]

Inheritance basics

Definition. `class Sub extends Super` makes Sub inherit all of Super's non-private members. Sub can also override methods (with `@Override`) and add new ones.

```
public class Animal {
    protected String name;
    public void greet() { System.out.println("hi"); }
}

public class Dog extends Animal {
    @Override
    public void greet() {
        System.out.println("woof, I'm " + name);
    }
}
```

super

- `super.method(args)` → calls the parent's version (skips override).
- `super(args)` → calls parent constructor; must be the FIRST statement of subclass constructor.

Dynamic vs Static type

Definition. Static type = compile-time type (declared). Dynamic type = runtime type (actual object).

Plain English. `Animal a = new Dog();` — static type `Animal`, dynamic type `Dog`.

Method dispatch rule (memorize). For overridden methods (instance methods), Java uses **dynamic dispatch** — calls the method based on dynamic type. For overloaded methods (different signatures), it uses **static dispatch** — picks the version based on static type.

```
Animal a = new Dog();
a.greet();           // calls Dog.greet() — DYNAMIC dispatch
```

equals and hashCode contract

Two-rule contract (must hold together):

1. If `a.equals(b)` then `a.hashCode() == b.hashCode()`.
2. If `a.hashCode() == b.hashCode()`, no guarantee about `a.equals(b)`.

Canonical equals template.

```

@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (o == null || getClass() != o.getClass()) return false;
    Dog other = (Dog) o;
    return Objects.equals(this.name, other.name);
}

@Override
public int hashCode() {
    return Objects.hash(name);
}

```

Iterable & Iterator

Plain English. Implementing `Iterable<T>` lets you use `for (T x : myObj)`. Need:

1. Class implements `Iterable<T>` and provides `iterator()`.
2. Returned `Iterator<T>` has `hasNext()` and `next()`.

```

public class SLList<T> implements Iterable<T> {
    @Override
    public Iterator<T> iterator() {
        return new SLListIterator();
    }

    private class SLListIterator implements Iterator<T> {
        private IntNode p = sentinel.next;

        @Override
        public boolean hasNext() { return p != null; }

        @Override
        public T next() {
            T item = p.item;
            p = p.next;
            return item;
        }
    }
}

```

Generics

- `class Box<T>` declares a type parameter.
- `Box<Integer>` substitutes `Integer` for `T`.
- **Bounded:** `<T extends Comparable<T>>` says `T` must be comparable.
- **Type erasure:** at runtime, generics are erased to their bounds (or `Object`). You can't do `new T[10]`.

Exam patterns / pitfalls

- "What does this print?" — trace dynamic dispatch carefully when subclasses override.
 - Forgetting to override `hashCode()` after overriding `equals()` → broken `HashMap` behavior.
 - Static methods are NOT dispatched dynamically — `Super.f()` and `Sub.f()` are independent.
-

5. Asymptotics — Big-O / Ω / Θ

[Lec 13]

Definitions

- **$O(f(n))$** : upper bound (ceiling). The function grows AT MOST as fast as f .
- **$\Omega(f(n))$** : lower bound (floor). The function grows AT LEAST as fast as f .
- **$\Theta(f(n))$** : tight bound. The function grows EXACTLY as fast as f (up to constants).

Plain English. Multiple O bounds can be valid (any growth rate $\geq$ the truth). Only one Θ is valid (matching the truth). Always prefer Θ when possible.

Common growth rates (memorize ranking)

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n!$

Standard loop patterns

Loop	Iterations
<code>for (int i=0; i<n; i++)</code>	n
<code>for (int i=1; i<n; i*=2)</code>	$\log n$
<code>for (int i=n; i>0; i/=2)</code>	$\log n$
Nested: <code>for i<n { for j<n }</code>	n^2
Nested: <code>for i<n { for j<i }</code>	$n^2/2 = \Theta(n^2)$
<code>for i*=2 { for j<i }</code>	$1+2+4+\dots+n = \Theta(n)$ (geometric)

Method for any code-runtime question

1. Find the innermost line (or recursive call).
2. Count how many times it executes (write as a sum).
3. Simplify: drop constants, drop lower-order terms.
4. The dominant term is your Θ .

Common simplifications

- $\Theta(2n + 5) = \Theta(n)$ (drop constants and lower terms)
- $\Theta(n^2 + n) = \Theta(n^2)$
- $\Theta(\log_b n) = \Theta(\log n)$ for any constant base b
- $\Theta(\log n!) = \Theta(n \log n)$ (Stirling's approximation)

Exam patterns / pitfalls

- O is NOT worst case — you can talk about Big- O of any case (best, worst, average).
 - "Worst-case $O(n^2)$ " and "best-case $O(1)$ " are different statements about the same function.
 - Don't say " $\Theta(n^2)$ on average" if the function is sometimes faster — Θ requires a tight bound.
-

6. Recurrences and Master Theorem

[Lec 15–16]

Recurrence relation

Plain English. Recursive algorithm runtime described as a recurrence: $T(n)$ in terms of smaller inputs.

Common recurrences (memorize their solutions)

Recurrence	Solution
$T(n) = T(n-1) + O(1)$	$\Theta(n)$
$T(n) = T(n-1) + O(n)$	$\Theta(n^2)$
$T(n) = T(n/2) + O(1)$	$\Theta(\log n)$
$T(n) = T(n/2) + O(n)$	$\Theta(n)$
$T(n) = 2T(n/2) + O(1)$	$\Theta(n)$
$T(n) = 2T(n/2) + O(n)$	$\Theta(n \log n)$ — mergesort
$T(n) = 2T(n/2) + O(n^2)$	$\Theta(n^2)$
$T(n) = 2T(n-1) + O(1)$	$\Theta(2^n)$

Master Theorem

Statement. For $T(n) = a \cdot T(n/b) + f(n)$ with $a \geq 1, b > 1$:

1. If $f(n) = O(n^{(\log_b a - \epsilon)})$ for some $\epsilon > 0 \rightarrow T(n) = \Theta(n^{\log_b a})$ (work dominated by leaves)
2. If $f(n) = \Theta(n^{\log_b a}) \rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log n)$ (balanced)
3. If $f(n) = \Omega(n^{(\log_b a + \epsilon)})$ AND regularity condition holds $\rightarrow T(n) = \Theta(f(n))$ (work dominated by root)

Plain English. Compare $f(n)$ to $n^{\log_b a}$. The bigger one (asymptotically) wins; if they're equal, multiply by log.

Recursion tree method (when Master Theorem doesn't apply)

1. Draw the tree of recursive calls.
2. Compute work per level.
3. Sum across levels.

- Mergesort: each level does $\Theta(n)$ work, $\log n$ levels $\rightarrow \Theta(n \log n)$.
- Quicksort worst case: $T(n) = T(n-1) + \Theta(n) \rightarrow \Theta(n^2)$.
- Quicksort average: $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$.

Exam patterns / pitfalls

- Master Theorem: write down $a, b, f(n)$ clearly. Compute $n^{\log_b a}$. Compare.
 - " $T(n) = T(n-1) + \Theta(n)$ " $\rightarrow$ Master Theorem doesn't apply ($n-1$ isn't n/b). Use recursion tree: $1 + 2 + \dots + n = \Theta(n^2)$.
-

7. Amortized Analysis

[Lec 15]

Definition

Plain English. Average cost per operation over a *worst-case sequence* of operations. Used when individual operations vary in cost but average out.

Aggregate method (most common)

1. Find the total cost of any sequence of N operations.
2. Divide by N to get the amortized cost per operation.

Example: AList addLast (geometric resize)

- Start with capacity 1. Resize doubles whenever full.
- After N inserts, the resize work is $1 + 2 + 4 + \dots + N \approx 2N$.
- Plus N constant-time inserts.
- Total: $\Theta(N)$ work for N operations $\rightarrow \Theta(1)$ amortized per addLast.

Why doubling matters

- If you grew by $+1$ each resize: $1 + 2 + 3 + \dots + N = \Theta(N^2)$ total. $\rightarrow \Theta(N)$ amortized. Bad.
- If you grew by $\times 2$ each resize: $1 + 2 + 4 + \dots + N = \Theta(N)$ total. $\rightarrow \Theta(1)$ amortized. Good.

Banker's / Potential method (alternative framing)

Plain English. Pre-pay for expensive operations using "credits" earned during cheap ones. Each operation "costs" amortized cost; saved-up credits cover spikes.

Exam patterns / pitfalls

- "Amortized vs worst case" — single addLast can still be $\Theta(N)$ (the resize). Amortized is per-operation over a long sequence.
- The amortized analysis depends on the resize strategy; specify yours.

8. Disjoint Sets (Union-Find)

[Lec 14]

API

```
class DisjointSets {
    void connect(int p, int q);    // union: merge p's group with q's group
    boolean isConnected(int p, int q);
    int find(int p);              // return root of p's group
}
```

Implementations (evolution toward WQU + Path Compression)

1. Quick Find (id array)

- `id[i]` = group label of `i`.
- `isConnected`: `id[p] == id[q] → O(1)`.
- `connect`: scan all and rewrite labels → $O(n)$.

2. Quick Union (parent array)

- `parent[i]` = parent of `i`. `parent[i] == i` if `i` is root.
- `find(p)`: walk up parents until root. Up to $O(n)$ (skinny tree).
- `connect(p, q)`: `parent[find(p)] = find(q)`. $O(n)$ worst case.

3. Weighted Quick Union (WQU)

- Track size of each tree (typically as a negative number at root: `parent[root] = -size`).
- `connect`: attach smaller tree under larger. Tree height $\leq \log n$.
- All ops $O(\log n)$.

4. WQU + Path Compression

- During `find`, make every visited node point directly at the root.
- Amortized nearly $O(1)$ (technically $\alpha(n)$, inverse Ackermann — call it constant).

Canonical pseudocode (WQU + PC)

```
int[] parent;    // parent[i] = parent of i, OR -size if i is root

int find(int p) {
    int root = p;
    while (parent[root] >= 0) root = parent[root];

    // Path compression: make every node on the path point to root
    while (parent[p] >= 0) {
        int next = parent[p];
        parent[p] = root;
        p = next;
    }
    return root;
}
```

```

    parent[p] = root;
    p = next;
}
return root;
}

void connect(int p, int q) {
    int rp = find(p), rq = find(q);
    if (rp == rq) return;
    int sizeP = -parent[rp], sizeQ = -parent[rq];
    if (sizeP < sizeQ) {                // attach smaller under larger
        parent[rp] = rq;
        parent[rq] = -(sizeP + sizeQ);
    } else {
        parent[rq] = rp;
        parent[rp] = -(sizeP + sizeQ);
    }
}
}

```

Runtime table

Implementation	connect	isConnected
Quick Find	$\Theta(n)$	$\Theta(1)$
Quick Union	$\Theta(n)$ worst	$\Theta(n)$ worst
Weighted QU	$\Theta(\log n)$	$\Theta(\log n)$
WQU + Path Compression	$\Theta(\alpha(n)) \approx \Theta(1)$ amortized	$\Theta(\alpha(n)) \approx \Theta(1)$ amortized

Exam patterns / pitfalls

- `parent[]` size is fixed at construction. It doesn't shrink.
- "Trace WQU on these connect operations" → keep an array, mark roots negative with size.
- `connect(a, b)` modifies `parent[root_of_a]` (or `root_of_b`), NOT `parent[a]`.
- Path compression only fires during `find`, not during `connect` directly.

9. Binary Search Trees (BSTs)

[Lec 17]

Invariant

For every node N : all keys in N 's left subtree are $< N.key$; all keys in N 's right subtree are $> N.key$.

Operations

search(key): walk down using comparisons. $\Theta(h)$ where h is height.

insert(key): walk down to where the search would fall off; attach a new leaf. $\Theta(h)$.

delete(key) — Hibbard deletion:

- 0 children: just remove.
- 1 child: promote child.
- 2 children: find in-order successor (smallest in right subtree), replace, then delete successor (which now has at most 1 child).

Heights

- Best case (perfectly balanced): $h = \Theta(\log n)$
- Worst case (skinny / inserted in order): $h = \Theta(n)$
- $\rightarrow$ BSTs are NOT self-balancing; that's why we need RB-trees / B-trees.

In-order traversal (yields sorted output)

```
void inorder(Node n) {
    if (n == null) return;
    inorder(n.left);
    visit(n);
    inorder(n.right);
}
```

Runtime table (BST, h = height)

Op	Best	Worst
search	$\Theta(\log n)$	$\Theta(n)$
insert	$\Theta(\log n)$	$\Theta(n)$
delete	$\Theta(\log n)$	$\Theta(n)$
in-order traversal	$\Theta(n)$	$\Theta(n)$

Exam patterns / pitfalls

- "Trace BST after these inserts" — draw step by step, hang each new key as a leaf.
 - "What's the worst-case shape after inserting 1, 2, 3, 4, 5 in order?" — fully right-skewed, height $n-1$.
 - Hibbard deletion with 2 children: in-order successor (or predecessor), don't forget the recursive delete.
-

10. B-Trees (2-3-4 Trees)

[Lec 18]

Definition

A 2-3-4 tree is a B-tree with $L=3$ (each node holds 1 to 3 keys, has 2 to 4 children). All leaves at the SAME depth $\rightarrow$ tree is always perfectly height-balanced.

Invariants

1. Each non-leaf node with k keys has $k+1$ children.
2. Keys within a node are sorted.
3. Children's key ranges are bounded by their parent's keys (BST-like across the whole tree).
4. All leaves at the same depth.

Insertion (61A's reactive variant)

1. Walk down to the appropriate leaf (BST-style).
2. Insert the new key into that leaf.
3. If the leaf now has 4 keys (overfull): **split**.
 - Promote the **left-middle** key (the second-smallest of 4, per 61B convention) up to the parent.
 - Split the remaining 3 keys into two siblings of 1 and 2 keys (or however 61B defines it — verify slide 18).
4. If the parent now has 4 keys, split it too. Chain reaction up the tree.
5. The tree GROWS IN HEIGHT only when the root splits — creating a new root above.

Worked example (insert 1, 2, 3, 4 into empty 2-3-4 tree)

```
insert 1:           [1]
insert 2:           [1, 2]
insert 3:           [1, 2, 3]
insert 4:  [1, 2, 3] is overfull, split:
                    [2]
                   /  \
                  [1]  [3, 4]
```

Runtime

All operations: $\Theta(\log n)$. Tree height is always $\leq \log_2(n+1)$ for 2-3-4.

Exam patterns / pitfalls

- "Insert these keys, draw the tree" — most common B-tree question.

- "Which key gets promoted on split?" — 61B's convention is left-middle. Some textbooks use right-middle. Use the convention from your slides.
 - All leaves at same depth — if your trace produces uneven leaves, you made an error.
-

11. Tree Rotations and Red-Black Trees

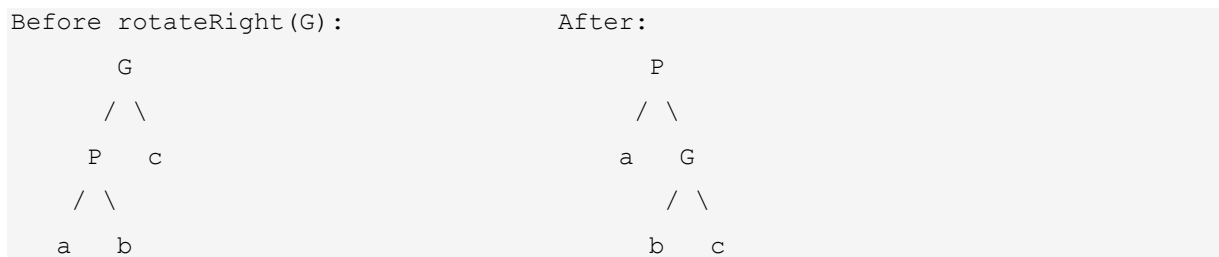
[Lec 19]

Tree Rotation

Definition. A rotation reshapes a BST while preserving the BST invariant. Two operations:

- `rotateLeft(G)`: G's right child becomes new root of subtree; G becomes left child.
- `rotateRight(G)`: G's left child becomes new root; G becomes right child.

rotateRight diagram



Rule. The "middle" subtree (b here) moves from being P's right child to being G's left child.

Red-Black Trees (LLRB convention)

Plain English. A BST that maintains balance using rotations and color-flips. Equivalent to a 2-3 (or 2-3-4) tree in structure.

Left-Leaning Red-Black (LLRB) invariants:

1. Every node is RED or BLACK.
2. Root is BLACK.
3. Red links lean left (right child can never be red on its own).
4. No two red links in a row.
5. Every path from root to NULL has the same number of black links (perfect black balance).

Mapping LLRB ↔ 2-3 Tree

- A red link in LLRB corresponds to two keys in the same 2-3-tree node.
- Black links separate 2-3-tree nodes.
- → an LLRB has the same height-balance properties as a 2-3 tree.

Insertion fix-ups (after standard BST insert with new node = RED)

1. If right child is RED and left is BLACK → `rotateLeft`.
2. If left is RED and left.left is RED → `rotateRight`.
3. If both children are RED → `flipColors` (both become BLACK, parent becomes RED).

Runtime

All ops: $\Theta(\log n)$. Tree height $\leq 2 \cdot \log n$ (factor of 2 because of red links).

Exam patterns / pitfalls

- "Trace insertion into an LLRB" — do BST insert, then apply the 3 fix-up rules bottom-up.
 - "Which rotation do I need here?" — left rotation if right child is red without a red left; right rotation if there are two red links in a row leaning left.
 - LLRB vs general RB: 61B uses LLRB; standard textbooks may use general red-black trees with different rules.
-

12. Heaps and Priority Queues

[Lec 20]

Min-Heap Invariant

1. Every node's key $\leq$ its children's keys.
2. The tree is a complete binary tree (filled level by level, last level filled left-to-right).

(Max-heap: parent $\geq$ children.)

Array Representation

A complete binary tree fits perfectly in an array indexed 1..n (root at index 1):

- Parent of i : $i / 2$
- Left child of i : $2 * i$
- Right child of i : $2 * i + 1$

(Some textbooks use 0-indexed; left= $2i+1$, right= $2i+2$, parent= $(i-1)/2$.)

swim (move a node up)

After inserting at the bottom: while the new node is smaller than its parent, swap.

```
void swim(int k) {
    while (k > 1 && a[k] < a[k/2]) {
        swap(k, k/2);
        k = k/2;
    }
}
```

sink (move a node down)

After replacing root with last element: while the node is larger than its smaller child, swap with that smaller child.

```
void sink(int k) {
    while (2*k <= n) {
        int j = 2*k;
        if (j < n && a[j+1] < a[j]) j = j + 1;    // pick smaller child
        if (a[k] <= a[j]) break;                // already in place
        swap(k, j);
        k = j;
    }
}
```

Operations

- `insert(x)`: append to end, then `swim`. $\Theta(\log n)$.
- `peekMin()`: return `a[1]`. $\Theta(1)$.
- `removeMin()`: save `a[1]`, replace with `a[n]`, decrement `n`, then `sink(1)`. $\Theta(\log n)$.
- `heapify(array)`: call `sink` on each non-leaf, bottom-up. $\Theta(n)$ (NOT $n \log n$).

Runtime table

Operation	Time
insert	$\Theta(\log n)$
peekMin	$\Theta(1)$
removeMin	$\Theta(\log n)$
heapify a whole array	$\Theta(n)$

Exam patterns / pitfalls

- "Insert these keys into an empty heap, draw the tree" — most common heap question. Use `swim`.
- "removeMin — show the array after each step" — replace root with last, decrement size, sink.
- Heapify in $\Theta(n)$ is counterintuitive — it's because the bottom layer (most nodes) only sinks $\Theta(1)$.
- Min-heap finds minimum in $\Theta(1)$, but finding the maximum requires $\Theta(n)$ (scan).

13. Hashing

[Lec 26–27]

Hash Map Definition

Plain English. Stores (key, value) pairs in a fixed-size array of "buckets." A hash function maps keys to bucket indices. Collisions handled by chaining (a list per bucket) or open addressing (probing).

Hash function

- Compute a key's `hashCode()` (an int).
- Reduce to bucket index: $\text{index} = (\text{hashCode} \% \text{capacity} + \text{capacity}) \% \text{capacity}$ (handle negative).

`equals/hashCode` contract (CRITICAL — see Inheritance section)

1. $a.\text{equals}(b) \Rightarrow a.\text{hashCode}() == b.\text{hashCode}()$.
2. The reverse is NOT required.

Separate chaining

Plain English. Each bucket holds a list of entries. Lookup walks the list to find a matching key.

- `put(k, v)`: hash to bucket, then walk list. If key exists, update; else prepend.
- `get(k)`: hash to bucket, walk list, return matching value.
- Average time: $\Theta(1 + \text{load factor})$.
- Worst case (all keys hash to same bucket): $\Theta(n)$.

Linear probing (open addressing)

Plain English. No lists — store entries directly in the array. On collision, probe to the next slot.

- `put(k, v)`: hash to index. If occupied with different key, check `index+1`, `+2`, ... wrapping around.
- `get(k)`: probe until match or empty slot.
- Tombstones needed for deletion.

Load factor and resize

- Load factor $\alpha = N / M$ (N entries, M buckets).
- When α exceeds a threshold (typically 0.5–0.75), double the array and rehash all entries.
- Resize is $\Theta(N)$, but amortized $\Theta(1)$ per insert (same argument as `ArrayList`).

Runtime table (assume good hash function, α bounded)

Operation	Average	Worst
-----------	---------	-------

put	$\Theta(1)$	$\Theta(n)$
get	$\Theta(1)$	$\Theta(n)$
remove	$\Theta(1)$	$\Theta(n)$
iteration over all entries	$\Theta(N + M)$	same

hashCode design rules

- Should distribute keys uniformly.
- Should be deterministic (same key $\rightarrow$ same code, every call).
- Must be consistent with equals.

```
@Override
public int hashCode() {
    int hash = 17;
    hash = 31 * hash + Objects.hashCode(field1);
    hash = 31 * hash + Objects.hashCode(field2);
    return hash;
}
```

Exam patterns / pitfalls

- "Insert these keys with this hash function — show the table" — common.
- "Calculate load factor; when does it resize?" — compute N/M .
- Wrong hashCode/equals consistency $\rightarrow$ hash maps lose entries.
- Linear probing performance degrades as load factor approaches 1 ("primary clustering"). Resize earlier than chaining would.

14. Tries

[Lec 28]

Definition

Plain English. A tree where each path from root to a node spells a string. Used for sets/maps with string keys, fast prefix queries.

Structure (simple R-way trie, R=26 for lowercase a–z)

```
class TrieNode {
    boolean isWord;           // true if a word ends here
    TrieNode[] children = new TrieNode[26];
}
```

Operations

insert(word):

```
void insert(String word) {
    TrieNode p = root;
    for (char c : word.toCharArray()) {
        int i = c - 'a';
        if (p.children[i] == null) p.children[i] = new TrieNode();
        p = p.children[i];
    }
    p.isWord = true;
}
```

contains(word):

```
boolean contains(String word) {
    TrieNode p = root;
    for (char c : word.toCharArray()) {
        int i = c - 'a';
        if (p.children[i] == null) return false;
        p = p.children[i];
    }
    return p.isWord;
}
```

Runtime

All operations on word of length L : $\Theta(L)$. Independent of N (number of words stored). This is the trie's superpower.

Prefix queries: finding all words with prefix P is $\Theta(L + \text{\#matches})$, trivial with a trie. Hash maps cannot do this efficiently.

Variants

- **R-way trie** — array of size R per node. Fast but space-wasteful for sparse alphabets.
- **TST (Ternary Search Trie)** — each node has left/middle/right child with character. More space-efficient. Verify whether 61B SP26 covers it.

Exam patterns / pitfalls

- "Insert these words; draw the trie." — node-by-node.
 - "What's the runtime of search?" — $\Theta(\text{length of word})$, NOT $\log n$.
 - Don't confuse with BST; tries don't use comparison-based search.
-

15. Graphs — Representations and Traversals (DFS, BFS)

[Lec 21–22]

Definitions

- **Vertex (node).** A point in the graph.
- **Edge.** A connection between two vertices. Can be directed or undirected; weighted or unweighted.
- **Path.** Sequence of edges.
- **Cycle.** Path that starts and ends at the same vertex.
- **Connected.** All vertices reachable from each other (undirected); strongly connected for directed.
- **DAG.** Directed Acyclic Graph.

Adjacency representations

Adjacency Matrix. $N \times N$ boolean (or weight) matrix. $A[i][j] = 1$ if edge $i \rightarrow j$.

- Space: $\Theta(V^2)$.
- Edge query: $\Theta(1)$.
- Iterate neighbors of i : $\Theta(V)$.

Adjacency List. Array of lists; $\text{adj}[i]$ = list of neighbors of i .

- Space: $\Theta(V + E)$.
- Edge query: $\Theta(\text{degree})$.
- Iterate neighbors of i : $\Theta(\text{degree})$.

When to use which: dense graphs ($E \approx V^2$) $\rightarrow$ matrix; sparse $\rightarrow$ list. Most real-world graphs are sparse $\rightarrow$ adjacency list.

DFS (Depth-First Search)

Pseudocode (recursive).

```
boolean[] visited;
void dfs(int v) {
    if (visited[v]) return;
    visited[v] = true;
    process(v);                // pre-order processing
    for (int u : adj[v]) dfs(u);
    // post-order processing (e.g., add to topo-sort stack)
}
```

Pseudocode (iterative with stack).

```
void dfs(int start) {
```

```

Stack<Integer> s = new Stack<>();
s.push(start);
while (!s.isEmpty()) {
    int v = s.pop();
    if (visited[v]) continue;
    visited[v] = true;
    process(v);
    for (int u : adj[v]) if (!visited[u]) s.push(u);
}
}

```

Key uses. Detecting cycles, topological sort, finding connected components, path finding (any path).

BFS (Breadth-First Search)

Pseudocode.

```

void bfs(int start) {
    Queue<Integer> q = new ArrayDeque<>();
    q.add(start);
    visited[start] = true;
    while (!q.isEmpty()) {
        int v = q.poll();
        process(v);
        for (int u : adj[v]) {
            if (!visited[u]) {
                visited[u] = true;
                q.add(u);
            }
        }
    }
}
}

```

Key uses. Shortest path in UNWEIGHTED graphs, level-by-level processing.

Runtime

Both DFS and BFS: $\Theta(V + E)$ with adjacency list, $\Theta(V^2)$ with adjacency matrix.

DFS vs BFS choice

- DFS: simpler recursive code, good for "explore everything," topological order, cycle detection.
- BFS: gives shortest path in unweighted graphs (number of edges).

Exam patterns / pitfalls

- "Trace DFS / BFS visit order" — depends on neighbor ordering. The exam will tell you ("explore neighbors in order shown" or "in alphabetical order").
 - DFS does NOT find shortest paths. BFS does (in unweighted graphs).
 - Forgetting to mark `visited` causes infinite loops on cyclic graphs.
-

16. Shortest Paths — Dijkstra, A*

[Lec 23]

Single-Source Shortest Path

Goal. Given source s , find shortest path from s to every other vertex.

Dijkstra's Algorithm

Requires. All edge weights ≥ 0 . (Fails with negative edges.)

Pseudocode.

```
dist[s] = 0; dist[other] = INF;
PriorityQueue<Vertex> pq;      // min-heap by dist
pq.add(s);

while (!pq.isEmpty()) {
    Vertex v = pq.poll();      // unvisited vertex with smallest dist
    if (visited[v]) continue;
    visited[v] = true;
    for (Edge e : v.edges) {
        Vertex u = e.to();
        double newDist = dist[v] + e.weight();
        if (newDist < dist[u]) {
            dist[u] = newDist;
            pred[u] = v;
            pq.add(u);          // (could decrease-key; usually re-add)
        }
    }
}
```

Invariant. When a vertex is polled, its `dist` is final (it's the true shortest distance from s).

Why this works. Because edges are non-negative, you can never improve a finalized vertex by going through later vertices.

Runtime

- With binary heap: $\Theta((V + E) \log V)$.
- With Fibonacci heap (theoretical): $\Theta(V \log V + E)$.
- 61B's standard answer: **$\Theta(E \log V)$** (assumes connected, $E \geq V$).

Why Dijkstra fails on negative edges

Once a vertex is finalized, Dijkstra ignores it. A negative edge could later improve it, but Dijkstra won't revisit. (Use Bellman-Ford for negative edges; not in 61B SP26 scope.)

A* Search

Plain English. Dijkstra prioritizes by $\text{dist}[v]$. A* prioritizes by $\text{dist}[v] + h(v)$ where $h(v)$ is a heuristic estimate of remaining distance to the goal.

- If h is **admissible** (never overestimates true remaining distance), A* finds the optimal path.
- If $h \equiv 0$, A* reduces to Dijkstra.
- A* is much faster when a good heuristic is available (e.g., Euclidean distance for road networks).

Exam patterns / pitfalls

- "Trace Dijkstra step-by-step" — most common graph question. Maintain (visited, dist, pred) table; pull smallest dist each step.
 - "Why does Dijkstra fail with negative edges?" — show a 3-vertex counterexample.
 - A* heuristic must be admissible AND (for graph search) consistent.
-

17. Minimum Spanning Trees — Prim, Kruskal

[Lec 24]

MST Definition

A spanning tree of a connected, weighted, undirected graph that has minimum total edge weight.

Cut Property (key MST theorem)

Statement. For any cut (partition of vertices into two sets), the minimum-weight edge crossing the cut is in the MST.

Plain English. Both Prim and Kruskal exploit this — they greedily add a minimum crossing edge at each step.

Prim's Algorithm

Plain English. Grow a single tree. Start with one vertex. Repeatedly add the cheapest edge connecting the tree to an outside vertex.

Pseudocode.

```
PriorityQueue<Edge> pq;           // edges crossing the current tree boundary
boolean[] inTree;
inTree[start] = true;
add all edges from start to pq;

while (treeSize < V) {
    Edge e = pq.poll();           // smallest crossing edge
    Vertex v = the unvisited endpoint of e;
    if (inTree[v]) continue;     // both endpoints in tree → skip
    inTree[v] = true;
    add e to MST;
    add v's edges to pq (skip those whose other endpoint is already in tree);
}
```

Runtime. $\Theta(E \log V)$ with binary heap.

Kruskal's Algorithm

Plain English. Don't grow a tree — sort all edges, add them in order, skip any that would create a cycle. Use Union-Find to detect cycles.

Pseudocode.

```
sort edges by weight ascending;
DisjointSets ds = new DisjointSets(V);
```

```

List<Edge> mst = new ArrayList<>();

for (Edge e : sortedEdges) {
    if (!ds.isConnected(e.u, e.v)) {
        ds.connect(e.u, e.v);
        mst.add(e);
        if (mst.size() == V - 1) break;
    }
}

```

Runtime. $\Theta(E \log E)$ for sorting + $\Theta(E \alpha(V))$ for union-find $\approx \Theta(E \log V)$.

Comparison

Aspect	Prim	Kruskal
Approach	Grow tree from one vertex	Add edges by weight; skip cycles
Helper	Priority queue	Sort + union-find
Best for	Dense graphs	Sparse graphs
Runtime	$\Theta(E \log V)$	$\Theta(E \log E)$

Exam patterns / pitfalls

- "Trace Prim's / Kruskal's on this graph" — maintain MST edge set step by step.
- Tie-breaking on equal-weight edges is exam-specified. Read carefully.
- Both algorithms produce A minimum spanning tree, possibly different trees if there are ties (same total weight).
- Prim with a Fibonacci heap is asymptotically faster but not in 61B scope.

18. Directed Acyclic Graphs and Topological Sort

[Lec 25]

DAG (Directed Acyclic Graph)

Definition. A directed graph with no cycles.

Plain English. Models dependencies (course prerequisites, build orders, scheduling).

Topological Order

Definition. A linear ordering of vertices such that for every directed edge $u \rightarrow v$, u appears before v .

Plain English. "Process A before B if there's a path from A to B." Only exists for DAGs.

Algorithm 1 — DFS-based topological sort

Plain English. Run DFS. As each vertex finishes (post-order), push it onto a stack. The stack, popped, gives a topological order.

```
Stack<Integer> order;
boolean[] visited;
void dfs(int v) {
    visited[v] = true;
    for (int u : adj[v]) if (!visited[u]) dfs(u);
    order.push(v);           // post-order
}

void topoSort() {
    for (int v = 0; v < V; v++) if (!visited[v]) dfs(v);
    // pop everything from order → topological order
}
```

Runtime. $\Theta(V + E)$.

Algorithm 2 — Kahn's algorithm (BFS-based)

Plain English. Repeatedly remove vertices with in-degree 0.

```
int[] indegree = compute in-degrees;
Queue<Integer> q;
for (int v = 0; v < V; v++) if (indegree[v] == 0) q.add(v);

while (!q.isEmpty()) {
    int v = q.poll();
```

```

    output v;                                // next in topo order
    for (int u : adj[v]) {
        if (--indegree[u] == 0) q.add(u);
    }
}

```

Runtime. $\Theta(V + E)$.

Cycle detection with topological sort

- DFS-based: a back edge during DFS = a cycle.
- Kahn's: if you can't process all vertices (queue empties before output reaches V), there's a cycle.

Critical / Longest Path in DAG

Plain English. In project scheduling, the longest path in a DAG of tasks gives the minimum total time.

Algorithm. Topologically sort; relax edges in topological order computing longest distance.

Exam patterns / pitfalls

- "Give a topological order" — multiple valid answers. State the order; if the exam pre-specifies tie-breaking, follow it.
 - "Does this graph have a topological order?" — only if it's a DAG (no cycles).
 - DFS-based: the order is the REVERSE of finishing order.
-

19. Sorting — Basic Sorts, Quicksort, Mergesort

[Lec 32–34]

Selection Sort

Plain English. Find the smallest, swap to front. Repeat for the rest.

```
for (int i = 0; i < n; i++) {
    int minIdx = i;
    for (int j = i+1; j < n; j++)
        if (a[j] < a[minIdx]) minIdx = j;
    swap(a, i, minIdx);
}
```

Runtime. $\Theta(n^2)$ always (best, average, worst). Stable: NO. In-place: yes.

Insertion Sort

Plain English. Walk left-to-right; for each element, slide it back into the sorted prefix.

```
for (int i = 1; i < n; i++) {
    int x = a[i], j = i - 1;
    while (j >= 0 && a[j] > x) {
        a[j+1] = a[j]; j--;
    }
    a[j+1] = x;
}
```

Runtime. Best $\Theta(n)$ (already sorted), worst $\Theta(n^2)$, average $\Theta(n^2)$. Stable: YES. In-place: yes.

Used in practice. For small n , beats fancier sorts due to low overhead. Often combined with quicksort/mergesort for the base case.

Mergesort

Plain English. Split array in half, recursively sort each half, then merge.

```
void mergesort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int mid = (lo + hi) / 2;
    mergesort(a, lo, mid);
    mergesort(a, mid+1, hi);
    merge(a, lo, mid, hi);           // merge two sorted subarrays
}
```

Runtime. Always $\Theta(n \log n)$ (best, average, worst). Stable: YES. In-place: NO ($\Theta(n)$ extra space).

Recurrence. $T(n) = 2T(n/2) + \Theta(n) \rightarrow \Theta(n \log n)$.

Quicksort

Plain English. Pick a pivot. Partition: smaller-than-pivot left, bigger-than-pivot right. Recurse on each side.

```
void quicksort(int[] a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);    // returns final index of pivot
    quicksort(a, lo, p - 1);
    quicksort(a, p + 1, hi);
}
```

Lomuto partition (pivot = last):

```
int partition(int[] a, int lo, int hi) {
    int pivot = a[hi];
    int i = lo;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) { swap(a, i, j); i++; }
    }
    swap(a, i, hi);
    return i;
}
```

Runtime.

- Best/average: $\Theta(n \log n)$
- Worst: $\Theta(n^2)$ — happens when pivot is always min or max (e.g., already sorted array with pivot=last)
- Stable: NO (depends on partition; standard variants are not).
- In-place: yes.

Pivot strategies.

- First/last: simple but susceptible to sorted-input worst case.
- Random pivot: avoids adversarial worst case in expectation.
- Median-of-three: pick median of first/middle/last; reduces likelihood of bad pivot.

Quickselect (find k-th smallest)

Plain English. Like quicksort, but only recurse on the side containing the k-th element.

Runtime. Average $\Theta(n)$, worst $\Theta(n^2)$. With median-of-medians: worst $\Theta(n)$ (theoretical).

Heapsort

Plain English. Heapify the array; repeatedly extract the max.

Runtime. $\Theta(n \log n)$ always. In-place: yes. Stable: NO.

Sorting summary table (memorize)

Algorithm	Best	Average	Worst	Space	Stable?
Selection	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	No
Insertion	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Yes
Mergesort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n)$	Yes
Quicksort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(\log n)$ stack	Usually No
Heapsort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(1)$	No
Radix (LSD)	$\Theta(WN)$	$\Theta(WN)$	$\Theta(WN)$	$\Theta(N + R)$	Yes

(W = number of digits, R = radix size for Radix Sort.)

Stability — what it means

Plain English. Two records with equal keys retain their original relative order. Matters when sorting by multiple keys (sort by name, then stable-sort by age — names stay in order within each age).

Exam patterns / pitfalls

- "Show the array after each pass of insertion sort" — outer loop iterations.
- "Show the partition step in quicksort with pivot=X" — important to specify partition variant; results differ.
- "Which sort is best for nearly-sorted data?" — insertion sort ($\Theta(n)$).
- "Which sort needs $O(n)$ extra space?" — mergesort.

20. Sorting — Theoretical Bounds, Radix Sort

[Lec 35–36]

Theoretical Lower Bound for Comparison Sorts

Theorem. Any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case.

Argument (decision tree). A comparison sort can be modeled as a binary decision tree where each leaf is a permutation. There are $n!$ permutations. A binary tree of depth d has $\leq 2^d$ leaves, so we need $2^d \geq n!$, i.e., $d \geq \log_2(n!) \approx n \log n$ (Stirling).

Implication. Mergesort, heapsort are asymptotically optimal among comparison sorts.

Radix Sort (LSD — Least Significant Digit)

Plain English. Sort by the rightmost digit, then by the next-to-rightmost, etc. Each digit-pass is a stable counting sort.

Why it beats the $n \log n$ bound. Radix sort doesn't compare elements directly — it inspects digits. The lower bound only applies to comparison sorts.

Pseudocode (LSD on integers, base R).

```
void radixSort(int[] a, int W) {      // W = number of digits per number
    for (int d = 0; d < W; d++) {
        countingSort(a, d, R);        // stable-sort by d-th digit
    }
}
```

Runtime. $\Theta(W \cdot (N + R))$ where N is array size, R is radix (alphabet size), W is max number of digits. For fixed-width integers, W is constant $\rightarrow \Theta(N)$.

MSD vs LSD

- **LSD.** Start at least significant digit. Stable counting sort each pass. Simple, requires fixed-width keys.
- **MSD.** Start at most significant digit. Recursive (each prefix becomes a sub-problem). Useful for variable-length strings.

Counting Sort (the building block)

Plain English. For small-range integer keys ($0..K$), count occurrences of each key, then output in order.

```
int[] count = new int[K+1];
for (int x : a) count[x]++;
int idx = 0;
for (int k = 0; k <= K; k++)
    for (int j = 0; j < count[k]; j++)
        a[idx++] = k;
```

Runtime. $\Theta(N + K)$. Stable variant requires prefix-sum + back-to-front placement.

Exam patterns / pitfalls

- "Trace LSD radix sort on these integers" — pass by pass, by digit.
 - "Why is radix sort not used for floating-point keys?" — interpreting bits is tricky; comparison-based usually wins for floats.
 - "Why doesn't radix violate the $n \log n$ lower bound?" — it doesn't compare; it inspects bits.
-

21. Compression (Huffman) — light coverage

[Lec 39]

Huffman Coding

Plain English. Variable-length prefix-free codes for symbols, where frequent symbols get shorter codes.

Algorithm.

1. Compute frequency of each symbol.
2. Build a min-heap of (frequency, symbol) pairs.
3. Repeatedly: pop the two smallest, merge into a new internal node with frequency = sum, push back.
4. The final tree's root-to-leaf paths give the codes (left=0, right=1).

Runtime. $\Theta(K \log K)$ where K is number of distinct symbols.

Why it works. Optimal among prefix-free codes: minimizes total bits for the given frequency distribution (Shannon-Fano-Elias / Huffman optimality).

Exam patterns / pitfalls

- "Build the Huffman tree for these frequencies" — typical question.
 - "Why prefix-free?" — so that any encoded bit string can be decoded uniquely without delimiters.
 - Generally a small section on the final, if it appears at all.
-

Closing notes

This guide spans every major topic from MT1 to the final. Use it together with:

- Your `MT2_study_guide.md` and `MT2_cheatsheet.md` (sections 5–12 here are the same scope; defer to those when they trigger).
- Past finals: Spring 2025 + Fall 2024 + Fall 2023 (representative recent ones).
- The course site: <https://sp26.datastructur.es/> for exam logistics.

Verify before exam day:

- Final exam exact time and location at <https://sp26.datastructur.es/>.
- Cheat sheet allowance (count of double-sided handwritten sheets).
- Any topics you missed in lecture (sp26 specifics).

Memorize cold (cheat sheet candidates):

- Operation runtime tables for: lists (SLList/DLList/AList), BSTs, heaps, hash maps, graph algorithms, all sorts.
- Master Theorem statement and the $n^{\log_b a}$ comparison rule.
- Common recurrence solutions table (Section 6).
- Heap array-indexing formulas (parent = $i/2$, children = $2i$ and $2i+1$).
- Disjoint Sets WQU+PC operations and $\sim \Theta(1)$ amortized claim.
- B-tree (2-3-4) split-and-promote rule.
- LLRB three fix-up rules (rotateLeft / rotateRight / flipColors).
- Dijkstra invariant ("polled vertex has final dist") and the non-negative-edge requirement.
- Prim vs Kruskal selection criteria.
- Topological sort: DFS post-order vs Kahn's in-degree.
- Quicksort partition (Lomuto or Hoare — pick one and know it cold).
- Sorting summary table (best/avg/worst/space/stable).
- Comparison-sort lower bound argument (decision tree $\rightarrow \Theta(n \log n)$).